Worksheet: Understanding CLIP

This worksheet helps you understand the core concepts of CLIP (Contrastive Language-Image Pre-Training).

Learning objectives:

- Understand how text and images are encoded into a shared space.
- Implement the core logic for creating positive and negative pairs using a similarity matrix.
- Implement the symmetric contrastive loss function.
- Visualize the alignment between images and text.

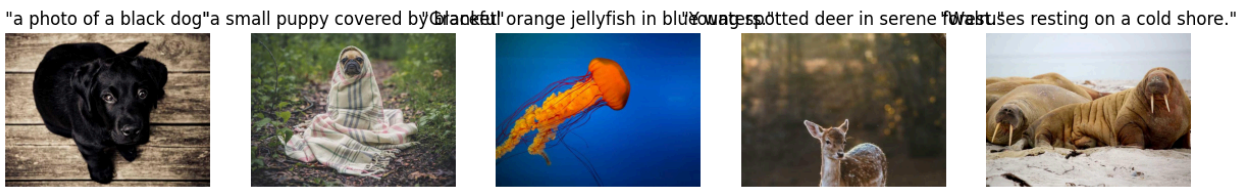
In [ ]:
```
## 1) Preliminaries — Install Packages

# We need the 'transformers' library from Hugging Face to easily load CLIP,
# 'torch' for computation, 'Pillow' for images, and 'matplotlib' for visualiza

!pip install -q transformers torch torchvision matplotlib Pillow requests

# Python imports and device detection
import os
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import numpy as np
import requests
from PIL import Image
from transformers import CLIPProcessor, CLIPModel

print('PyTorch:', torch.__version__)

# Set device (GPU if available, otherwise CPU)
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

# Make sure a small folder exists for images
os.makedirs('images_clip', exist_ok=True)
```

```
/home/yf25843/anaconda3/envs/qunat/lib/python3.9/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See htt
ps://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
PyTorch: 2.8.0+cu128
Device: cuda:0
```

In [21]:
```
## 2) Download 5 example images and define captions

# We will download 5 images and provide 5 matching text captions.
```

```python
# These will form our "batch" for this exercise.

# --- Image URLs ---
img_urls = [
    'https://picsum.photos/id/237/800/600', # A dog
    'https://picsum.photos/id/1025/800/600', # A puppy
    'https://picsum.photos/id/1069/800/600', # A beach/pier
    'https://picsum.photos/id/1003/800/600', # A deer
    'https://picsum.photos/id/1084/800/600'  # A cityscape
]

# --- Corresponding Text Captions ---
captions = [
    "a photo of a black dog",
    "a small puppy covered by blanket",
    "Graceful orange jellyfish in blue waters.",
    "Young spotted deer in serene forest.",
    "Walruses resting on a cold shore."
]

# --- Download the images ---
filenames = []
for i, url in enumerate(img_urls):
    r = requests.get(url)
    fname = f'images_clip/img_{i+1}.jpg'
    with open(fname, 'wb') as f:
        f.write(r.content)
    filenames.append(fname)
    print('Saved', fname)

# --- Display thumbnails and captions ---
plt.figure(figsize=(15, 4))
for i, (fname, caption) in enumerate(zip(filenames, captions)):
    img = Image.open(fname).convert('RGB')
    plt.subplot(1, 5, i+1)
    plt.imshow(img)
    plt.axis('off')
    plt.title(f'"{caption}"')
plt.suptitle('Our Image-Text Pairs', fontsize=14)
plt.show()
```

```
Saved images_clip/img_1.jpg
Saved images_clip/img_2.jpg
Saved images_clip/img_3.jpg
Saved images_clip/img_4.jpg
Saved images_clip/img_5.jpg
```



In [22]:
```
## 3) Load CLIP Model and Processor

# We will use the base model from OpenAI, loaded via Hugging Face.
model_name = "openai/clip-vit-base-patch32"

# The CLIPProcessor handles both image preprocessing (resizing, normalizing)
# and text tokenization.
processor = CLIPProcessor.from_pretrained(model_name)

# The CLIPModel contains the two towers:
# 1. model.get_text_features()
# 2. model.get_image_features()
model = CLIPModel.from_pretrained(model_name).to(device)

print("CLIP Processor and Model loaded successfully.")
```

```
CLIP Processor and Model loaded successfully.
```

In [23]:
```
## 4) Get Image and Text Embeddings (A single batch)

# First, load all our downloaded images into a list of PIL Images
pil_images = [Image.open(fname).convert('RGB') for fname in filenames]

# --- TODO: STUDENT TASK 1 ---
# Process the images and text captions using the `processor`.
# We want to get PyTorch tensors back.
# Reference: processor(text=..., images=..., return_tensors="pt", padding=True
#
# After processing, move the inputs to the correct `device`.

# 1. Process inputs
inputs = processor(text=captions, images=pil_images, return_tensors="pt", padd
inputs = {k: v.to(device) for k, v in inputs.items()} # Move to device


# --- TODO: STUDENT TASK 2 ---
# Pass the processed inputs through the `model` to get the embeddings.
# Make sure to wrap this in a `with torch.no_grad():` block.
# The model will return an object, and we want to get:
# - image_embeds = outputs.image_embeds
# - text_embeds = outputs.text_embeds

# 2. Get embeddings
with torch.no_grad():
```

```
    outputs = model(**inputs)
    image_embeds = outputs.image_embeds
    text_embeds = outputs.text_embeds

# --- TODO: STUDENT TASK 3 ---
# L2-Normalize the embeddings. This is crucial for cosine similarity.
# The .norm() function will be useful here.
# 3. L2-Normalize
image_embeds_norm = image_embeds / image_embeds.norm(dim=-1, keepdim=True)
text_embeds_norm = text_embeds / text_embeds.norm(dim=-1, keepdim=True)


# Print the resulting shapes
print(f"Image embeddings shape: {image_embeds_norm.shape}") # Should be (N, 51
print(f"Text embeddings shape: {text_embeds_norm.shape}")   # Should be (N, 51
```

```
Image embeddings shape: torch.Size([5, 512])
Text embeddings shape: torch.Size([5, 512])
```

In [24]:
```
## 5) Core Logic: Building Positive/Negative Pairs (Similarity Matrix)

# The core idea of CLIP is to create a similarity matrix between *all*
# images and *all* text captions in the batch.

# --- TODO: STUDENT TASK 4 ---
# Calculate the cosine similarity matrix between the *normalized* text and ima
# Since they are L2-normalized, cosine similarity is just a matrix multiplicat
# We want a matrix `similarity` of shape (N_text, N_images).
# Hint: You will need to transpose (.T) one of the embedding matrices.

similarity = text_embeds_norm @ image_embeds_norm.T

# The `similarity` matrix now holds all our pairs:
# - The DIAGONAL (i, i) contains the similarity for POSITIVE pairs (correctly
# - The OFF-DIAGONAL (i, j) contains the similarity for NEGATIVE pairs (mismat

print(f"Similarity matrix shape: {similarity.shape}")
```

```
Similarity matrix shape: torch.Size([5, 5])
```

In [25]:
```
## 6) Core Logic: Calculating the Contrastive Loss

# CLIP uses a temperature-scaled, symmetric cross-entropy loss.
# We scale the similarities (logits) by a temperature parameter (model.logit_s
# Then, we calculate the loss in two directions:
# 1. Image-to-Text: For each image, predict the correct text.
# 2. Text-to-Image: For each text, predict the correct image.

# --- TODO: STUDENT TASK 5 ---
# Implement the symmetric loss calculation.

# 1. Scale the logits by the model's learned temperature parameter
#    (Hint: it needs to be exponentiated: .exp())
logit_scale = model.logit_scale.exp()
```

```
logits = similarity * logit_scale

# 2. Create the "ground truth" labels. For a batch of size N,
#    the correct pair for item 0 is 0, for 1 is 1, etc.
#    (Hint: Use torch.arange(N) and move it to the device)
N = similarity.shape[0]
labels = torch.arange(N).to(device)

# 3. Calculate the Image-to-Text loss (loss_i)
#    - Transpose the logits to (N_images, N_text)
#    - Calculate cross_entropy between these logits and the labels
logits_per_image = logits.T
loss_i = F.cross_entropy(logits_per_image, labels)

# 4. Calculate the Text-to-Image loss (loss_t)
#    - Use the original logits (N_text, N_images)
#    - Calculate cross_entropy between these logits and the labels
logits_per_text = logits
loss_t = F.cross_entropy(logits_per_text, labels)

# 5. Calculate the final symmetric loss
total_loss = (loss_i + loss_t) / 2.0


# Print the losses
print(f"Image-to-Text Loss: {loss_i.item():.4f}")
print(f"Text-to-Image Loss: {loss_t.item():.4f}")
print(f"Total Symmetric Loss: {total_loss.item():.4f}")
```

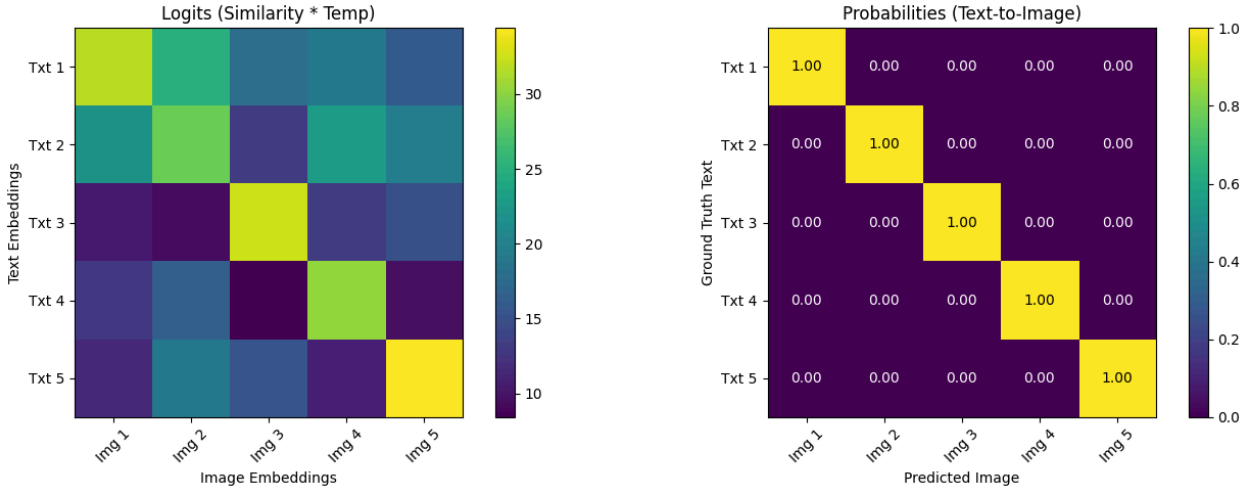
```
Image-to-Text Loss: 0.0042
Text-to-Image Loss: 0.0009
Total Symmetric Loss: 0.0026
```

In [26]:

```
## 7) Visualize the Similarity Matrix and Probabilities

# This cell is fully implemented.
# It visualizes the logits and the resulting probabilities (after softmax)
# to show how the model aligns images and text.

# We need the logits and labels from the previous cell.
if 'logits_per_text' in globals() and 'logits_per_image' in globals():

    # Calculate probabilities
    probs_t_i = F.softmax(logits_per_text, dim=1).cpu().detach().numpy()
    probs_i_t = F.softmax(logits_per_image, dim=1).cpu().detach().numpy()

    # --- Plot 1: Raw Logits (Similarity) ---
    plt.figure(figsize=(14, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(logits.cpu().detach().numpy(), cmap='viridis')
    plt.title("Logits (Similarity * Temp)")
    plt.xlabel("Image Embeddings")
    plt.ylabel("Text Embeddings")
    plt.xticks(range(N), [f"Img {i+1}" for i in range(N)], rotation=45)
```

```
    plt.yticks(range(N), [f"Txt {i+1}" for i in range(N)])
    plt.colorbar()
    print("Diagonal (Positive Pairs):", [f"{logits[i,i].item():.2f}" for i in
    print("Notice the diagonal should have the highest values (brightest squar


    # --- Plot 2: Probabilities (Text-to-Image) ---
    plt.subplot(1, 2, 2)
    plt.imshow(probs_t_i, cmap='viridis', vmin=0, vmax=1)
    plt.title("Probabilities (Text-to-Image)")
    plt.xlabel("Predicted Image")
    plt.ylabel("Ground Truth Text")
    plt.xticks(range(N), [f"Img {i+1}" for i in range(N)], rotation=45)
    plt.yticks(range(N), [f"Txt {i+1}" for i in range(N)])
    plt.colorbar()

    # Add text for probabilities
    for i in range(N):
        for j in range(N):
            plt.text(j, i, f"{probs_t_i[i, j]:.2f}", ha='center', va='center',

    plt.tight_layout()
    plt.show()

else:
    print("Could not find 'logits_per_text'. Please run the previous cells fir
```

```
Diagonal (Positive Pairs): ['31.70', '28.70', '32.48', '30.32', '34.41']
Notice the diagonal should have the highest values (brightest squares).
```



8) How to Run & Next Steps

1. Upload this notebook to Google Colab (`File -> Upload notebook`).
2. Enable GPU (`Runtime -> Change runtime type -> GPU`).
3. Execute cells 1-4 to install packages, download data, and load the model.

4. Fill in the `TODO` sections in cells 5, 6, and 7 to implement the core logic.
5. Run all cells. The visualization in cell 8 will show you how well the (pretrained) CLIP model matches your images and captions. You should see a bright diagonal!

Experiment ideas:

- Try changing the captions in Cell 3 to be *incorrect* (e.g., `captions = ["a car", "a tree", "a dog", "a cat", "a person"]`). Rerun the notebook and see how the similarity matrix and loss change.
- Try using very similar images (e.g., 5 different pictures of dogs) and see if the model can still distinguish them based on subtle text differences.

9) Example: CLIP for Zero-Shot Image Classification

One of the most powerful features of CLIP is "Zero-Shot" classification.

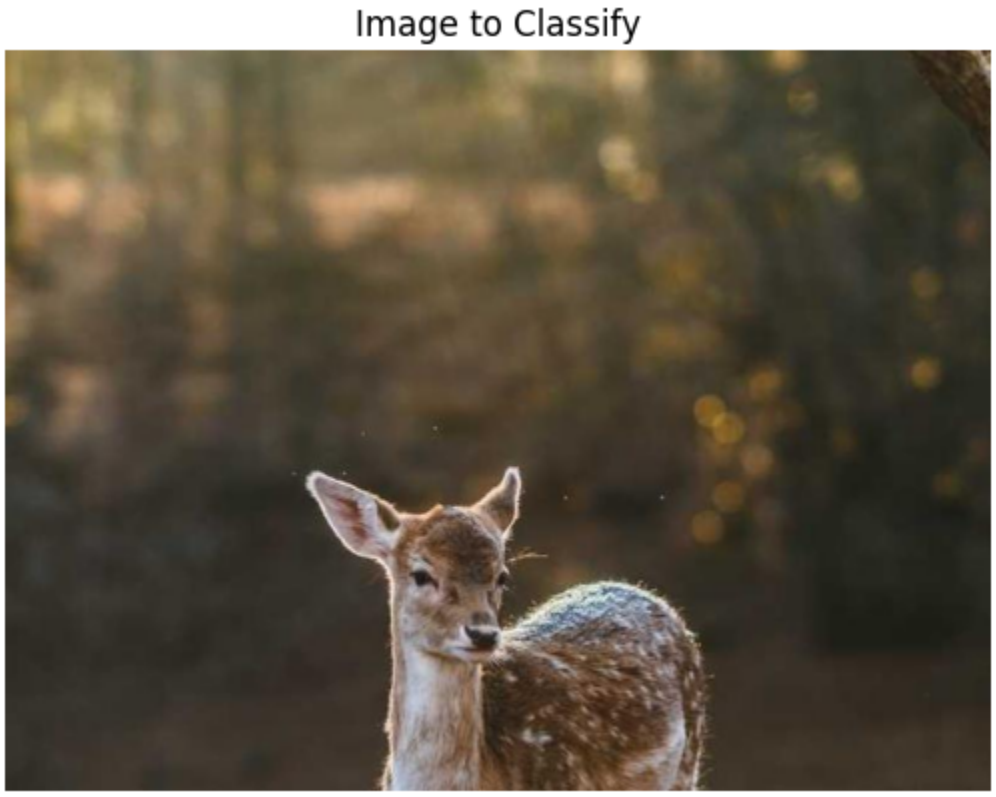
This means we can classify an image into categories the model has **never** seen during its training, just by providing the text descriptions for those categories.

In this example, we'll use one of our previously loaded images (the deer) and see if the model can correctly identify it from a new set of candidate labels.

In [27]:
```
## 10) Prepare Image and Text Labels for Zero-Shot Classification

# 1. We'll select one image from our downloaded list (img_4.jpg is the deer)
# The `filenames` list was created in Cell 3
image_to_classify_path = filenames[3]
image_pil = Image.open(image_to_classify_path).convert('RGB')

# 2. Define our "zero-shot" candidate categories
# Note: The model was not specifically trained on these exact labels!
# We can even add some distractor labels.
class_labels = [
    "a photo of a dog",
    "a photo of a cat",
    "a photo of a deer",
    "a photo of a car",
    "a photo of a bird"
]

print(f"Preparing to classify this image: {image_to_classify_path}")
plt.imshow(image_pil)
plt.title("Image to Classify")
```

```
plt.axis('off')
plt.show()
```

```
Preparing to classify this image: images_clip/img_4.jpg
```



In [28]:
```
## 11) Process the Image and Text Labels

# 1. Use the processor (loaded in Cell 4) to handle our *one* image and *multi
# Note: We put the single image in a list: [image_pil]
inputs = processor(
    text=class_labels,
    images=[image_pil],
    return_tensors="pt",
    padding=True
)

# 2. Move the inputs to the device (GPU/CPU)
inputs = {k: v.to(device) for k, v in inputs.items()}

print("Image and text processed.")
```

```
Image and text processed.
```

In [29]:
```
## 12) Get Embeddings and Calculate Similarities

# 1. Get the image and text embeddings from the model (loaded in Cell 4)
with torch.no_grad():
    outputs = model(**inputs)
    image_embeds = outputs.image_embeds # Shape: (1, 512)
    text_embeds = outputs.text_embeds   # Shape: (5, 512)
```

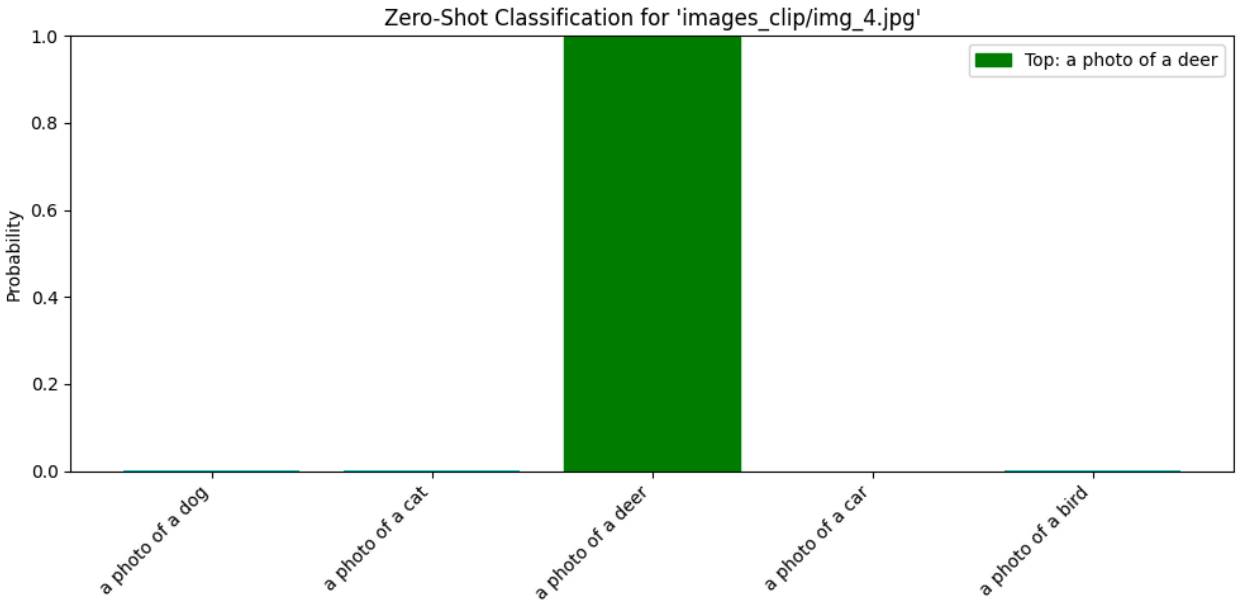
```
# 2. L2 Normalize the embeddings
image_embeds_norm = image_embeds / image_embeds.norm(dim=-1, keepdim=True)
text_embeds_norm = text_embeds / text_embeds.norm(dim=-1, keepdim=True)

# 3. Calculate logits (cosine similarity * temperature)
# We multiply the (1, 512) image embedding by the (512, 5) transposed text emb
logit_scale = model.logit_scale.exp()
logits = (image_embeds_norm @ text_embeds_norm.T) * logit_scale

# 4. Convert logits to probabilities using Softmax
probs = F.softmax(logits, dim=1)

print(f"Logits (1 image vs 5 texts) shape: {logits.shape}")
print(f"Probabilities shape: {probs.shape}")
```

```
Logits (1 image vs 5 texts) shape: torch.Size([1, 5])
Probabilities shape: torch.Size([1, 5])
```

In [31]:

```
## 13) Visualize the Zero-Shot Classification Results

# Convert probabilities to a numpy array for plotting
probs_cpu = probs.detach().cpu().numpy().squeeze() # Shape (5,)
num_classes = len(class_labels)

# Print results
print("--- Classification Results ---")
for i, label in enumerate(class_labels):
    print(f"{label}: {probs_cpu[i]*100:.2f}%")

# Find the highest scoring index
top_index = np.argmax(probs_cpu)
top_label = class_labels[top_index]
top_score = probs_cpu[top_index]

print(f"\n--> Top Prediction: '{top_label}' (Score: {top_score:.4f})")

# Plot as a bar chart
plt.figure(figsize=(10, 5))
bars = plt.bar(range(num_classes), probs_cpu, color='c')

# Highlight the top prediction in green
bars[top_index].set_color('g')
bars[top_index].set_label(f"Top: {top_label}")

plt.title(f"Zero-Shot Classification for '{image_to_classify_path}'")
plt.xticks(range(num_classes), class_labels, rotation=45, ha='right')
plt.ylabel("Probability")
plt.ylim(0, 1)
plt.legend()
plt.tight_layout()
plt.show()
```

```
--- Classification Results ---
a photo of a dog: 0.04%
a photo of a cat: 0.01%
a photo of a deer: 99.93%
a photo of a car: 0.00%
a photo of a bird: 0.02%

--> Top Prediction: 'a photo of a deer' (Score: 0.9993)
```



In [ ]: